CS5730 Operating Systems — Midterm Cheat Sheet

Built from past exams 2011, 2013, 2019, 2020, 2021, 2022 plus lecture topics 01–08. The professor repeats questions **almost word for word** (2021 and 2022 are nearly identical). Read the question, find it in §1, write the model answer. Section numbers: §1 repeated questions · §2 sync code · §3 scheduling · §4 process/thread · §5 memory · §6 deadlock · §7 basics.

§0 Exam intel: what gets tested

Format: about 8–9 essay questions, 100 pts, "answer concisely (3–4 sentences)". Sub-parts are worth 5–20 pts. Code questions ask you to *write or modify* pseudo-C. Gantt charts are expected for real-time scheduling.

Topic (count of exams it appeared in, out of 6)	Years	§
Multilevel queue: list 2 inter-queue schemes + modify so a process moves queues (MLFQ) ×5	11,13,19,21,22	1.6
Why disabling interrupts is bad for user-level sync ×4	11,13,21,22	1.1
Scheduling-criteria conflicts (5 pairs) ×4	11,13,20,21	1.5
Monitors: Hoare vs signal-and-continue, queues, semaphore impl, dining philosophers ×4	11,19,22(×2)	1.12–1.14
Busy waiting + semaphore with block/wakeup ×3	13,21,22	1.2
fork()/thread_create counting ×3	13,19,20	1.8
exec() before fork() ×3; fork/exec in a multithreaded process ×3	19,20,21 / 13,19,20	1.7
Processor affinity ×3	19,20,21	1.9
Regressive round-robin (favors CPU-bound) ×3	13,19,20	1.10
Bounded buffer: BUFFER_SIZE−1 ×2; empty/full semaphores ×1	21,22 / 13	1.3
RMS vs EDF Gantt (p1=50,t1=25,p2=75,t2=30) ×2	21,22	1.4
Fix the 100-pthreads program ×2; lottery scheduling ×2	19,20	1.11, 1.10
Paging address-bit math ×2; Intel IA-32 translation; demand-paging EAT	11,22; 20; 20	1.16–1.18
Peterson proof + variant; TestAndSet/Swap (+ fails if not atomic); wait/signal must be atomic	11; 13; 11	1.15
Thread models (many-to-one on MP; many-to-many with k kernel threads); socket ports; race condition (bank)	11,13; 21; 13	1.19–1.21

Not seen in past midterms but in the slides: deadlock / banker's (topic 07). A short reference is in §6, just in case.

§1 Repeated questions with model answers

1.1 Why is disabling interrupts NOT appropriate for user-level synchronization (even on 1 CPU)? ×4

- Disabling interrupts is a **privileged instruction**. Letting user code run it breaks dual-mode protection.
- A buggy or malicious program could disable interrupts and **never re-enable them**. Then the **timer interrupt** can't fire, the OS never regains control, there is no preemption or context switch, and one process **monopolizes the CPU** (system hangs).
- While interrupts are off, I/O completions and clock ticks are delayed or lost (time-of-day drifts, devices sit idle) and responsiveness suffers.
- It also doesn't scale: on a multiprocessor it only affects the local CPU, so it gives no mutual exclusion there anyway.

Use hardware atomic instructions (test_and_set/swap) or OS-provided semaphores/mutexes instead.

1.2 (a) What is busy-waiting? How does it hurt performance (or deadlock)? (b) Semaphore without busy waiting (block/wakeup) ×3

(a) Busy waiting (a **spinlock**) means a process waiting to enter its critical section loops continuously testing a condition, e.g. `while(test_and_set(&lock));` or `while(S<=0);`. It **wastes CPU cycles** that other processes could use. On a uniprocessor the spinner burns its whole time slice, and the lock holder can't run to release the lock in the meantime. **Deadlock:** with priority scheduling, if a high-priority process spins waiting for a lock held by a low-priority process, the low one never gets scheduled to release it, so both wait forever (priority-inversion style). Spinning is fine only when expected waits are shorter than a context switch (on multiprocessors).

(b)

```
typedef struct {
    int value;
    struct process *list;   /* waiting queue (PCBs) */
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();            /* suspend: running -> waiting */
    }
}
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);          /* waiting -> ready queue */
    }
}
```

A negative value means |value| processes are waiting. block() and wakeup() are basic system calls. wait() and signal() must themselves be atomic (a CS). On a uniprocessor, disable interrupts inside them; on a multiprocessor, use a spinlock inside them. Busy waiting is **not fully eliminated**, but it is limited to the very short wait/signal code, so it is rare and brief.

1.3 Bounded buffer: why only BUFFER_SIZE−1 slots? / What do `empty` and `full` do? ×3

N−1: the only shared state is `in` and `out`, and **empty is defined as `in == out`**. If the producer were allowed to fill all N slots, `in` would wrap around to equal `out`, and "full" would look exactly like "empty". So the producer stops when `(in+1)%N == out`, leaving one slot always unused to tell full from empty. (E.g. N=10: in=9, out=0 → (9+1)%10==0 → full with only 9 items.) Fix: add a `counter` of items (which then needs synchronization because `counter++` / `--` race) or a full/empty flag.

Semaphores: `mutex=1` gives mutual exclusion on buffer access. `empty=n` counts empty slots: the producer does `wait(empty)` before inserting (it blocks when the buffer is full) and the consumer does `signal(empty)` after removing. `full=0` counts filled slots: the consumer does `wait(full)` (it blocks when empty) and the producer does `signal(full)`. They replace busy-wait loops and let all n slots be used.

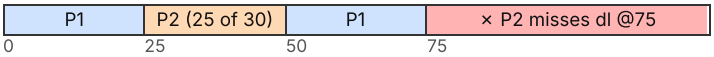
```
Producer:                       Consumer:
do { produce item;              do { wait(full);
  wait(empty);                         wait(mutex);
  wait(mutex);                         remove item from buffer;
  add item to buffer;                  signal(mutex);
  signal(mutex);                       signal(empty);
  signal(full);                        consume item;
} while(1);                     } while(1);
```

The order matters: doing wait(mutex) before wait(empty) can deadlock when the buffer is full.

1.4 P1: p=d=50, t=25; P2: p=d=75, t=30. (a) RMS? (b) EDF? ×2 (20 pts)

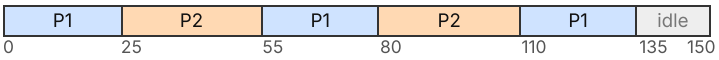
U = 25/50 + 30/75 = 0.50 + 0.40 = **0.90**. The RMS bound for N=2 is $2(2^{1/2}-1)$ = **0.828**. U > bound, so RMS is not guaranteed; check with a chart. (U ≤ 1, so EDF works.)

(a) RMS: priority is static and the shorter period wins, so **P1 > P2**.



At t=50, P1's 2nd instance preempts P2, which has only had 25 of its 30 ms. P1 runs 50–75, and at t=75 P2 has missed its deadline. **Not schedulable by RMS.**

(b) EDF: dynamic priority, the earliest absolute deadline runs (ties: don't preempt).



- 0: P1(dl 50) vs P2(dl 75): P1 runs 0–25. Then P2 runs from 25.
- 50: P1#2 arrives (dl 100), but P2's dl 75 is earlier, so **P2 keeps running** and finishes at 55. P1 runs 55–80.
- 75: P2#2 arrives (dl 150). P1's dl 100 is earlier, so P1 finishes at 80. P2 runs 80–110.
- 100: P1#3 arrives (dl 150), tied with P2, so no preemption. P1 runs 110–135, then idle 135–150.
- LCM = 150, so the pattern repeats. **All deadlines met.**

1.5 How do these scheduling criteria conflict? ×4 (15 pts)

- **CPU utilization vs response time:** utilization is highest when context switches are rare (FCFS, long quanta). Good response time needs frequent preemption (short RR quanta), and each switch is overhead that lowers useful CPU utilization.
- **Avg turnaround vs max waiting time:** SJF/SRTF minimizes *average* turnaround by always running short jobs first, but long jobs can **starve**, so the *maximum* waiting time becomes very large (unbounded).
- **I/O device utilization vs CPU utilization:** to keep devices busy you must run I/O-bound jobs as soon as their I/O completes (preempting CPU-bound jobs), which causes many context switches. Maximizing CPU use means letting CPU-bound jobs run without switching, which leaves devices idle.
- **Deadline vs fairness:** meeting deadlines means always giving the CPU to the most urgent (real-time / earliest-deadline) task, possibly starving others, which is unfair. Fair (equal-share) scheduling can make urgent tasks miss deadlines.
- **Throughput vs context-switch overhead:** high throughput often means preempting to finish many short jobs (SJF, small quantum), and every preemption adds context-switch overhead. Minimizing switches (FCFS) leaves short jobs stuck behind long ones (convoy effect), which lowers throughput.
- *(2013)* **CPU utilization vs throughput:** running long CPU-bound jobs keeps the CPU 100% busy but completes few jobs per unit time. Favoring many short or I/O-bound jobs completes more jobs but adds switches and idle I/O waits, which lowers utilization.

1.6 Multilevel queue (MLQ): (a) list 2 ways to schedule among queues; (b) modify so a new process automatically lands in the right queue after a few runs ×5

(a) (1) **Fixed-priority preemptive**: serve all of the higher queue (e.g. foreground) before any of the lower one (background), which can cause starvation. (2) **Time slice among queues**: each queue gets a fixed share of CPU time, e.g. 80% foreground with RR and 20% background with FCFS.

(b) Turn it into a **multilevel FEEDBACK queue**: let processes **move between queues** based on observed CPU-burst behavior.

```
            new system proc
                  v
Q0 system process        (highest, fixed, FCFS/priority)
 new faculty proc v
Q1 faculty interactive   RR q=8   --used full q--> demote to Q3
                  ^-------- blocks for I/O early: promote ----+
 new student proc v                                          |
Q2 student interactive   RR q=8   --used full q--> demote to Q4|
                  ^-------- blocks for I/O early: promote ----+
Q3 faculty batch         RR q=16 (or FCFS)
Q4 student batch         FCFS      (lowest)
+ aging: a process waiting too long in Q3/Q4 moves up
```

- A new process enters the **interactive** queue for its class (faculty or student) with a small quantum.
- If it **uses its whole quantum**, it looks CPU-bound and is **demoted** to the batch queue (bigger quantum, lower priority).
- If it **blocks for I/O before the quantum expires**, it looks interactive or I/O-bound and stays or is **promoted**.
- **Aging** promotes processes that wait too long, which prevents starvation. System processes stay fixed at the top.
- The MLFQ is defined by 5 parameters: number of queues; the algorithm for each queue; when to **upgrade**; when to **demote**; which queue a new process enters.

Why distinguish I/O-bound from CPU-bound (2013)? I/O-bound processes have short CPU bursts. Giving them priority keeps I/O devices busy and interactive response fast, while CPU-bound jobs fill the CPU gaps. That gives better overall CPU *and* device utilization, and the long-term scheduler wants a good mix.

Linux support for both inter-queue schemes (2011): fixed priority between classes: **real-time** priorities 0–99 (SCHED_FIFO / SCHED_RR) always run before **normal** tasks (nice −20..+19 maps to 100–139). Time slicing: the O(1) scheduler gives each task a quantum (higher priority gets a longer one) using **active/expired arrays**, and swaps them when all slices are used. CFS (2.6.23+) gives each task a **proportion** of CPU by **vruntime** within a target latency.

1.7 exec() before fork()? / fork() and exec() in a multithreaded process ×3 each

exec before fork: exec() **replaces the calling process's memory image** (code, data, stack) with the new program and **does not return** on success. So the fork() after it is never reached: no child is created, and the original program is gone, replaced by the exec'd program running under the same PID. (fork() only runs if exec fails and returns −1.)

One thread calls fork(): it depends on the system. Some UNIXes have two versions: one duplicates **all threads**, the other (POSIX default) duplicates **only the calling thread**. If exec() is called right after fork(), duplicating only the caller is enough (the other threads would be thrown away anyway). If the child doesn't exec, duplicating all threads may be needed.
One thread calls exec(): the program in exec's parameter **replaces the entire process, including ALL threads**.

1.8 How many unique processes / threads? ×3

```
pid = fork();            /* (1) P0 -> C1                      */
if (pid == 0) {          /* only C1                           */
    fork();              /* (2) C1 -> C2                      */
    thread_create(...);  /* C1 and C2 each create 1 thread    */
}
fork();                  /* (3) P0,C1,C2 each fork -> +3      */
```

(a) 6 processes (P0, C1, C2 + 3 from the last fork; that's 5 new if you exclude the original, so state which you mean). **(b) 2 threads** created by thread_create (one in C1, one in C2). The final fork copies only the calling thread, so no extra threads. This is the textbook answer and got full credit (graded 10/10 in 2020).

Rule of thumb: n unconditional sequential forks give 2^n processes. Draw the tree and label each fork.

1.9 Processor affinity: what is it and why consider it on multi-CPU systems? ×3

Processor affinity means a process prefers to keep running on the **processor it is currently running on**. While it runs, its data fills that CPU's **cache**. If it migrates, the old cache contents must be **invalidated** and the new CPU's cache **repopulated**, which is expensive (cache misses, memory stalls). So SMP schedulers try to avoid migration. **Soft affinity:** the OS tries to keep the process on the same CPU but doesn't guarantee it. **Hard affinity:** the process specifies the set of CPUs it may run on (processor sets). On **NUMA** machines, affinity also keeps the process near its local memory. Trade-off: affinity works against **load balancing** (push/pull migration).

1.10 Regressive round-robin: which type of process does it favor? / Lottery scheduling: how do you favor high priority? ×3 / ×2

Regressive RR favors CPU-bound (batch) processes. A process that uses its **entire quantum** (CPU-bound) is rewarded with +10 ms of quantum (up to 100 ms) **and a priority boost**. A process that **blocks before its quantum ends** (I/O-bound / interactive) is penalized with −5 ms and **gets no priority boost**. Over time CPU-bound processes get longer slices and higher priority.

Lottery (BTV OS): give **more lottery tickets to higher-priority threads**. A thread's chance of winning each 20 ms draw is (its tickets)/(total tickets), so a thread holding k of T tickets gets about k/T of the CPU over time (proportional share). Priority can also be adjusted dynamically by giving or taking tickets.

1.11 Program that should create 100 pthreads printing 1..100. Correct? Fix it. ×2

Incorrect. (1) Every thread is passed **the same address** `&param1`, and main keeps changing param1 in the loop. Threads read it whenever they get scheduled, so they print duplicates or wrong values (often lots of 100s), not 1..100. (2) main never **pthread_join**s, so it can return or exit before the threads run, which kills them. (3) `*y` dereferences a `void*` and needs a cast.

```
void *doSomething(void *y) {
    printf("%d\n", *(int *)y);
    pthread_exit(0);
}
int main() {
    pthread_t t1[100];
    int param[100], i;                /* one slot per thread */
    for (i = 0; i < 100; i++) {
        param[i] = i + 1;
        pthread_create(&t1[i], NULL, doSomething, &param[i]);
    }
    for (i = 0; i < 100; i++)
        pthread_join(t1[i], NULL);    /* wait for all */
    return 0;
}
```

Answers that only added join scored 7/10, so the shared-address bug is the main point. If they must print *in order*, the threads also need synchronization, or each thread can be joined right after it's created (which serializes them).

1.12 Monitors: Hoare (signal-and-wait) diagram → depict signal-and-continue 2019, 2022 (10–15 pts)

Only one process is active in the monitor. P (=B) calls `q.signal()` while Q (=A) is in `q.wait()`.

Signal-and-wait (Hoare): A resumes **immediately**, and B blocks in the **signal queue** until A leaves or waits. **Signal-and-continue (Mesa):** B **keeps running**, and A moves to a ready/re-entry queue until B leaves or waits.

#	Hoare (given)	Signal & continue (answer)
1	A enters monitor	A enters monitor
2	A waits on q → wait queue	A waits on q → q's wait queue (releases monitor)
3	B enters monitor	B enters monitor
4	B signals; **B** enters signal queue	B signals; **A** moves from wait queue → signal (re-entry) queue; **B continues**
5	A re-enters monitor	B leaves monitor
6	A leaves monitor	A re-enters monitor (signal queue has priority over entry queue)
7	B re-enters monitor	A leaves monitor
8	B leaves monitor	Another process admitted from entry queue
9	Another admitted from entry queue	—

```
Signal & continue picture:
  Entry Queue --1(A),3(B)--> [ MONITOR ] --5: B exits, 7: A exits-->
  2: A calls q.wait  : A -> Wait Queue(q), monitor released
  4: B calls q.signal: A  Wait Queue(q) -> Signal Queue; B keeps runn
  6: monitor free    : A  Signal Queue  -> back in monitor (before en
```

Comparison: Hoare guarantees the condition still holds when A resumes, so `if(...) x.wait()` is enough, but it costs extra context switches. With S&C the state may change before A runs, so use **`while(!cond) x.wait()`**. S&C is simpler and cheaper, and it's what Java and pthreads use.

1.13 Monitor queue states (2011): entry: P1,P2; signal queue: P3; active P0; x: P4,P5; y: P6. Signal-and-wait. Each event is independent.

Event	Active	Entry	Signal q	x	y
P7 accesses M	P0	P1,P2,**P7**	P3	P4,P5	P6
P0: x.signal()	**P4**	P1,P2	P3,**P0**	P5	P6
P0: y.wait()	**P3**	P1,P2	—	P4,P5	P6,**P0**

When the monitor frees up, the signal queue has priority over the entry queue. (Under signal-and-continue, x.signal would leave P0 active, move P4 to the signal/ready queue → P3,P4, and leave x: P5.)

1.14 Implement the monitor with SIGNAL-AND-CONTINUE via semaphores (modify the given signal-and-wait code) 2022 (20 pts), 2019 (20 pts)

Given (signal-and-wait): mutex=1, next=0, next_count=0. Each procedure F: `wait(mutex); body; if(next_count>0) signal(next); else signal(mutex);`. In this version, `next` holds suspended *signalers*.
x.wait: `x_count++; if(next_count>0) signal(next); else signal(mutex); wait(x_sem); x_count--;`
x.signal: `if(x_count>0){ next_count++; signal(x_sem); wait(next); next_count--; }`

Answer: the signaler must NOT block. Now `next` / `next_count` hold *signaled processes waiting to re-enter* (the signal queue).

```
semaphore mutex = 1;    // monitor lock
semaphore next  = 0;    // signaled procs waiting to RE-ENTER
int next_count  = 0;
/* per condition x */
semaphore x_sem = 0;  int x_count = 0;

Each procedure F:                 (UNCHANGED)
  wait(mutex);
  ... body of F ...
  if (next_count > 0) signal(next); // pass monitor to signaled proc
  else                signal(mutex);

x.wait:
  x_count++;
  if (next_count > 0) signal(next);   // release monitor
  else                signal(mutex);
  wait(x_sem);        // sleep until someone signals x
  wait(next);         // NEW: wait till signaler leaves/waits
  next_count--;       // NEW: back inside the monitor

x.signal:              // signal-and-continue
  if (x_count > 0) {
      x_count--;      // NEW: signaler decrements (no double wake)
      next_count++;   // the woken proc will wait on next
      signal(x_sem);  // wake it; NO wait(next): keep running
  }
```

Key changes: (1) removed `wait(next)` from x.signal, so the signaler continues. (2) The waiter, after being woken, does `wait(next)` to re-acquire the monitor when the signaler exits or waits, which gives it priority over the entry queue. (3) x_count is decremented by the signaler, so two signals in a row can't wake the same waiter twice. Simpler acceptable variant (no priority): `x.signal: if(x_count>0){x_count--; signal(x_sem);}`; `x.wait: x_count++; signal(mutex); wait(x_sem); wait(mutex);`. Callers must use `while(!cond) x.wait();`.

2019: Complete dining philosophers using the monitor with S&C via semaphores:

```
enum {THINKING, HUNGRY, EATING} state[5];
semaphore mutex = 1, next = 0;  int next_count = 0;
semaphore self_sem[5] = {0};    int self_count[5] = {0};

void enter() { wait(mutex); }
void leave() { if (next_count > 0) signal(next); else signal(mutex);
void cwait(int i) {                     /* self[i].wait()   */
    self_count[i]++;
    leave();                            /* release monitor  */
    wait(self_sem[i]);
    wait(next);  next_count--;          /* re-enter          */
}
void csignal(int i) {                   /* self[i].signal() */
    if (self_count[i] > 0) {
        self_count[i]--; next_count++;
        signal(self_sem[i]);            /* continue running */
    }
}
void test(int i) {                      /* called inside monitor */
    if (state[(i+4)%5] != EATING && state[i] == HUNGRY &&
        state[(i+1)%5] != EATING) {
        state[i] = EATING;
        csignal(i);
    }
}
void pickup(int i) {
    enter();
    state[i] = HUNGRY;
    test(i);
    while (state[i] != EATING) cwait(i);   /* while: S&C-safe */
    leave();
}
void putdown(int i) {
    enter();
    state[i] = THINKING;
    test((i+4)%5);  test((i+1)%5);         /* wake neighbors */
    leave();
}
void init() { for (int i=0;i<5;i++) state[i]=THINKING; }
philosopher(i): while(1){ think(); pickup(i); eat(); putdown(i); }
```

This works with S&C because test() sets state[i]=EATING *before* signaling, and only philosopher i can change it back, so the condition still holds when i resumes. No deadlock; starvation is still possible.

1.15 Peterson proof; variant; TestAndSet / Swap; non-atomic wait/signal

```
flag[i] = TRUE;  turn = j;
while (turn == j && flag[j]) ;   /* busy wait */
   /* critical section */
flag[i] = FALSE;
```

- **Mutual exclusion:** Pi enters only if `flag[j]==F` or `turn==i`. If both were in the CS, both flags would be TRUE, so turn would have to equal both i and j at once, which is impossible. turn holds the *last* value written, and the process that wrote last waits.
- **Progress:** if Pj isn't trying (flag[j]=F), Pi enters immediately. If both try, turn picks one in finite time. Only processes in their entry sections affect the decision, never those in the remainder section.
- **Bounded waiting:** when Pj exits it sets flag[j]=F, so Pi enters. If Pj immediately re-requests, it sets turn=i, so Pi goes first. Pi waits for **at most one** entry of Pj.

Variant (entry `turn = i`, exit `turn = j`): **mutual exclusion is violated.** Trace: P0 sets flag0=T, turn=0, tests (turn==1?) false, and **enters**. Then P1 sets flag1=T, turn=1, tests (turn==0?) false, and **enters**. Both are in the CS. (A process that writes turn last never waits.)

```
/* test_and_set: atomic */
bool TAS(bool *t){bool rv=*t;
  *t=true; return rv;}
lock = false;
do { while (TAS(&lock)) ;
     /* CS */
     lock = false;
     /* remainder */
} while (1);
```

```
/* swap: atomic */
void swap(bool *a,bool *b){
  bool tmp=*a; *a=*b; *b=tmp;}
lock = false;  /* shared */
do { key = true;  /* local */
     while (key == true)
         swap(&lock, &key);
         /* CS */
     lock = false;
} while (1);
```

If TAS or Swap is not atomic: with lock=false, P0 reads lock (false), gets preempted, then P1 reads lock (false) and sets it true and enters the CS. P0 resumes, sets lock=true, and returns its earlier read value false, so it **also enters**. ME is violated. Swap fails the same way: both read lock=false into their keys before either writes true.

Non-atomic wait()/signal() (2011): S=1. P0 tests S<=0 (false), is preempted. P1 tests S<=0 (false), does S-- → 0 and enters the CS. P0 resumes, does S-- → −1 and enters too. ME is violated. (Similarly, two concurrent S++ can lose an update.)

Bounded-waiting TAS (n procs): `waiting[i]=T; key=T; while(waiting[i]&&key) key=TAS(&lock); waiting[i]=F; CS; j=(i+1)%n; while(j!=i && !waiting[j]) j=(j+1)%n; if(j==i) lock=F; else waiting[j]=F;`

1.16 Paging: 48-bit addressing, 4K pages, 64-bit CPU: max process memory? # of page-table entries? How to store the page table? 2022

- Max logical space = 2^{48} B = **256 TB**.
- Offset = 12 bits (4 KB = 2^{12}), so the page number has 48−12 = 36 bits, giving **2^{36} ≈ 68.7 billion (64 G) entries**.
- With 8-byte PTEs (64-bit), a flat table would be 2^{36}×8 = 2^{39} B = **512 GB per process**, which is impossible to keep contiguous in memory.
- **Store it hierarchically** ("page the page table"): x86-64 uses **4-level** paging with 9+9+9+9+12 bits. Each level's table has 2^9=512 entries × 8 B = 4 KB = exactly one page, and only the parts that are actually used get allocated. Alternatives: **hashed** page tables (common for >32-bit, sparse spaces) or an **inverted** page table (one entry per physical frame). Use a **TLB** to avoid the extra memory accesses.

2011: 64 pages × 1024 words, 32 frames. Logical address = log2(64)+log2(1024) = 6+10 = **16 bits**. Physical address = log2(32)+10 = 5+10 = **15 bits**.

Recipe: offset bits = log2(page size). Page # bits = logical bits − offset. # PTEs = $2^{\text{page\# bits}}$. Frame # bits = physical bits − offset. Table size = #PTE × PTE size. Two-level 32-bit/4 KB: 10|10|12. 32-bit/1 KB: p1=12, p2=10, d=10.

1.17 Intel 80386 (IA-32): CPU → logical addr → segmentation unit → linear addr → paging unit → physical addr 2020

Steps:

1. The CPU generates a **logical address = (selector, offset)**. The 16-bit selector has **s** = 13-bit segment index, **g** = 1 bit for GDT/LDT, and **p** = 2 bits for protection (privilege level).
2. The **segmentation unit** uses the selector to fetch the 8-byte **segment descriptor** from the LDT (private, up to 8K segments) or GDT (shared, 8K segments). Descriptors are cached in the segment registers. It checks offset ≤ limit and access rights (otherwise it traps), then computes **linear address = base + offset** (32-bit).
3. The **paging unit** splits the 32-bit linear address into **p1 (10) | p2 (10) | d (12)**. **CR3** points to the page directory. p1 indexes the page directory to get a page-table address. p2 indexes the page table to get the frame #. Then **physical = frame + d**. (For 4 MB pages the directory entry points straight at the frame: 10 | 22 offset.) The TLB caches translations; PAE adds a 3rd level for 36-bit physical addresses.

Advantages: segmentation gives the **user's logical view** (code, data, stack segments) with **per-segment protection and sharing**. Paging removes **external fragmentation**, allows noncontiguous allocation, and makes swapping and virtual memory easy. You get both protection levels and flexibility.

Disadvantages: complexity in hardware and OS. **Slower** translation: a segment lookup plus 2 page-table memory accesses per reference without TLB hits. More memory for tables (descriptor tables plus page tables). Paging still has internal fragmentation. Higher cost and power. That's why many architectures (RISC, ARM, x86-64 in practice with a flat model) use **paging only**.

1.18 Demand paging: fault service 8 ms (clean) / 20 ms (dirty, 70%), memory 100 ns. Max fault rate for EAT ≤ 200 ns? 2020

Avg service = 0.3×8 + 0.7×20 = 16.4 ms = 16,400,000 ns.
EAT = (1–p)(100) + p(16,400,000) ≤ 200 → 100 + 16,399,900·p ≤ 200 → **p ≤ 6.1×10^{-6}** (about 1 fault per 164,000 accesses).

General: EAT = (1–p)·ma + p·(fault overhead + [swap out] + swap in + restart). The 2020 student scored 2/10 by mixing up ms and ns. **Convert units!**

TLB EAT: EAT = α(ε+m) + (1–α)(ε+2m). The slides ignore ε: α=80%, m=100 gives 0.8×100 + 0.2×200 = **120 ns**; α=99% gives **101 ns**. Including ε=20: 0.8×120 + 0.2×220 = 140 ns.

1.19 Thread models on multiprocessors

Many-to-one on a multiprocessor vs a uniprocessor (2013): No real gain. All user threads map to **one kernel thread**, and the kernel schedules only that one, so only one user thread can run at a time on one CPU. There's no parallelism, and one blocking system call blocks all threads. (The other CPUs can only run other processes.)

Many-to-many, user threads > processors (2011):

- **Kernel threads < # processors:** some processors sit idle for this program because only that many threads can run in parallel. The hardware is under-used.
- **= # processors:** all CPUs can be used at once, but when a kernel thread blocks (I/O, page fault) its CPU goes idle, because there's no spare kernel thread to swap in.
- **> # processors but < # user threads:** best utilization. When one kernel thread blocks, another ready kernel thread runs on that CPU. The cost is some extra kernel-thread switching overhead.

Multithreading doesn't help (2011): a strictly **sequential** program where each step depends on the previous one (e.g. computing a tax return or a recurrence), or a CPU-bound single task on a uniprocessor. Threads add creation, switching, and synchronization overhead with no overlap to gain. Another example: a shell or program that only waits on one user.

1.20 Sockets: why port number instead of process ID? 2021

A **PID is assigned dynamically** and is only meaningful **within one host**. It changes every time the server restarts, and a remote client can't know it. A **port is a fixed, well-known number tied to a service** (HTTP 80, SSH 22; ports below 1024 are well-known), so clients can find the service without knowing which process provides it. The process behind a port can change (restart, failover, a pool of worker processes or threads), and one process can serve several ports. The socket = **IP:port** (e.g. 161.25.19.8:1625) identifies the endpoint independently of the OS's internal process numbering, and the network stack demultiplexes packets by port.

1.21 Race condition: shared account, husband withdraw() || wife deposit() 2013

`balance += amt` and `balance -= amt` are each load / modify / store. Example: balance=100. H reads 100 (withdraw 50). W reads 100 (deposit 50). H writes 50. W writes 150. **Final 150 instead of 100** (the other order gives 50): a lost update. The outcome depends on interleaving, which is a **race condition**. Fix: make both methods critical sections, e.g. `wait(mutex); balance ±= amt; signal(mutex);` (or a monitor / synchronized methods).
Same idea as the slide's counter++ / counter−− example: with counter=5, the interleaving S0–S5 can leave 4 or 6.

§2 Synchronization reference

Critical-section requirements

(1) **Mutual exclusion**: at most one process in its CS. (2) **Progress**: if no one is in the CS, the choice of who enters next involves only processes that want to enter and can't be postponed indefinitely. (3) **Bounded waiting**: there's a bound on how many times others can enter after a process has requested entry. Assume nonzero speed and nothing about relative speeds. Structure: entry section → CS → exit section → remainder.

Preemptive kernel: a process can be preempted in kernel mode (needs sync, more responsive). **Non-preemptive**: a process runs until it exits kernel mode, blocks, or yields, so the kernel is essentially race-free.

Semaphores

Busy-wait definition: `wait(S){while(S<=0); S--;}` `signal(S){S++;}` (P/V, atomic). **Counting** semaphores have an unrestricted range. **Binary** semaphores (0/1) are mutex locks. Binary from TAS: `wait(S){while(TAS(&S));}` `signal(S){S=0;}`. Mutual exclusion: `mutex=1; wait(mutex); CS; signal(mutex);`. Ordering (S1 before S2): `synch=0`; P1: `S1; signal(synch);` P2: `wait(synch); S2;`.

Deadlock with S=Q=1: P0 does wait(S), wait(Q); P1 does wait(Q), wait(S). **Starvation**: a process is never removed from the semaphore queue (e.g. LIFO). **Priority inversion**: low-priority P3 holds a lock that high-priority P1 needs, while medium-priority P2 preempts P3, so P1 waits on P2. Fix: **priority inheritance** (temporarily raise P3 to P1's priority).

Misuse: signal before wait breaks ME. wait, wait deadlocks. Omitting wait or signal breaks things. These bugs are hard to reproduce, which motivates monitors.

Readers–writers (1st variation: readers have priority, writers may starve)

```
semaphore rw_mutex = 1, mutex = 1;  int read_count = 0;
Writer: do { wait(rw_mutex); /*write*/ signal(rw_mutex); } while(1);
Reader: do { wait(mutex); read_count++;
             if (read_count == 1) wait(rw_mutex);  /* 1st locks */
             signal(mutex);
             /* read */
             wait(mutex); read_count--;
             if (read_count == 0) signal(rw_mutex);/* last unlocks */
             signal(mutex);
       } while(1);
```

2nd variation: once a writer is ready, it writes ASAP (readers may starve). Some kernels provide reader-writer locks.

Dining philosophers (semaphores)

```
semaphore chopstick[5] = {1,1,1,1,1};
do { wait(chopstick[i]); wait(chopstick[(i+1)%5]);
     /* eat */
     signal(chopstick[i]); signal(chopstick[(i+1)%5]);
     /* think */ } while(1);
```

Problem: deadlock if all 5 pick up their left chopstick at once. Remedies: (1) allow at most 4 philosophers at the table; (2) pick up both chopsticks only if both are free, inside a CS (the monitor solution); (3) asymmetric: odd philosophers take left then right, even ones take right then left.

Monitor basics

An ADT where only one process is active inside at a time. `condition x; x.wait()` suspends until `x.signal()`. `x.signal()` resumes exactly one waiter and **has no effect if nobody is waiting** (unlike a semaphore's signal, which is remembered). `x.wait(c)` is a conditional wait where the lowest c (priority number) is resumed first. The monitor DP solution (pickup/test/putdown) is in §1.14. It has **no deadlock, but starvation is possible**.

§3 CPU scheduling reference

Decisions happen when a process: (1) running→waiting, (2) running→ready, (3) waiting→ready, (4) terminates. Only (1) and (4) means **nonpreemptive**; otherwise it's **preemptive**. **Dispatcher:** context switch, switch to user mode, jump to the PC. Its delay is the **dispatch latency**.

Criteria: maximize CPU utilization and throughput (# completed per unit time). Minimize turnaround (submit→complete), waiting time (time in the ready queue), and response time (submit→first response).

Algorithm	Key points
FCFS	Nonpreemptive. Convoy effect (short jobs stuck behind long ones).
SJF / SRTF	Optimal minimum average waiting time. Needs burst prediction $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$. Long jobs can starve.
Priority	Starvation, fixed by **aging**. SJF is priority = predicted burst.
RR	Quantum q. Large q behaves like FCFS. Small q means high switch overhead. Rule of thumb: about 80% of bursts < q. Better response, often worse turnaround than SJF.
MLQ	Fixed queues (foreground RR, background FCFS). Between queues: fixed priority or time slice (80/20).
MLFQ	Processes move between queues. Ex: Q0 RR q=8 → Q1 RR q=16 → Q2 FCFS. A new job enters Q0 and is demoted if it doesn't finish its quantum.
Lottery / proportional share	T shares; an app with N shares gets N/T of the CPU.

Deterministic example (slides): P1=10, P2=29, P3=3, P4=7, P5=12, all arriving at 0. Avg wait: FCFS = (0+10+39+42+49)/5 = **28**. SJF (3,7,10,12,29) = (0+3+10+20+32)/5 = **13**. RR q=10 = **23**. Waiting = turnaround − burst; turnaround = completion − arrival.

Multiprocessor: asymmetric (one master CPU touches kernel data) vs **SMP** (each CPU self-schedules, with a common or private ready queue; most common). **Affinity** soft/hard (§1.9). **Load balancing**: **push** migration (a periodic task moves work off overloaded CPUs) and **pull** migration (an idle CPU pulls a waiting task). These work against affinity. Multicore with multiple hardware threads per core hides **memory stalls**. Virtualization: guests schedule without knowing they don't own the CPU, which hurts response time and clocks.

Real-time: soft (no guarantee) vs hard (must meet the deadline). **Interrupt latency** = interrupt arrival → ISR start. **Dispatch latency** = time to stop one process and start another; its conflict phase covers preempting kernel-mode processes and low-priority processes releasing resources that high-priority ones need. Periodic tasks have t (processing) ≤ d (deadline) ≤ p (period), and rate = 1/p. **RMS**: static priority by 1/period (shorter period = higher priority); it's optimal among static schemes; schedulable if $U = \Sigma\, t_i/p_i \le N(2^{1/N}-1)$ (N=1: 1, N=2: 0.828, N→∞: 0.69). Above that bound it might still be schedulable, so check a chart. **EDF**: dynamic priority by earliest deadline; feasible iff U ≤ 1 (theoretically); doesn't need periodic tasks. POSIX: **SCHED_FIFO** (FCFS, no time slicing among equal priorities) and **SCHED_RR** (time-sliced), via `pthread_attr_set/getsched_policy`.

RMS textbook example: P1(p=50,t=20), P2(p=100,t=35): U = 0.75, schedulable. P1(50,25), P2(80,35): U = 0.94, P2 misses its deadline at 80, but EDF meets all deadlines.

Linux: before 2.5, traditional UNIX. 2.5 introduced **O(1)**: preemptive priority, real-time 0–99 and nice 100–140, lower number = higher priority, **higher priority gets a larger quantum**, active/expired arrays per CPU runqueue. 2.6.23+ uses **CFS**: scheduling classes (real-time beats default), nice −20..+19, proportion of CPU via a **target latency** and per-task **vruntime** (the lowest vruntime runs next, kept in a red-black tree).
Windows: 32 priority levels: variable 1–15, real-time 16–31, 0 = memory-management thread. A thread runs until it blocks, its quantum expires, or a higher-priority thread preempts it. Quantum expiry lowers priority (never below the base). A wait boosts priority. The foreground window gets 3× the quantum.
Solaris: 6 classes (TS, IA, RT, SYS, FSS, FP). TS is an MLFQ with a dispatch table where a higher priority gets a smaller quantum.

Evaluation: deterministic modeling, queueing models (**Little's law n = λ·W**, e.g. λ=7/s and n=14 give W=2 s), simulation (trace tapes), implementation. **Amdahl:** speedup ≤ 1 / (S + (1−S)/N). With 25% serial on 2 cores: 1/(0.25+0.375) = 1.6×. As N→∞, the limit is 1/S.

§4 Processes, IPC, threads

Process = a program in execution (active). A program is passive (a file on disk). Memory layout: text, data (globals), heap (dynamic), stack (parameters, return addresses, locals). **States:** new → (admitted) ready → (dispatch) running → (interrupt) ready | (I/O or event wait) waiting → (completion) ready | (exit) terminated. **PCB:** state, PC, CPU registers, scheduling info, memory-management info, I/O status (open files), accounting. Linux uses `task_struct`.

Context switch: save state into PCB_{old} and load from PCB_{new}. It's pure overhead, and multiple register sets in hardware reduce it.
Queues: job queue (all processes), ready queue (in memory), device queues. **Long-term** (job) scheduler: infrequent, controls the **degree of multiprogramming**, aims for a good mix of I/O-bound (many short bursts) and CPU-bound (few long bursts) processes. **Short-term** (CPU) scheduler: runs every few milliseconds and must be fast. (Medium-term: swapping.)

fork() creates a child that copies the parent (new PID). It returns **0 in the child**, the **child's PID in the parent**, and −1 on error. **exec()** (execl, execlp uses PATH) replaces the memory image and doesn't return. **wait(&status)** blocks until a child ends and returns the child PID. **exit()**. A **zombie** has terminated but its parent hasn't called wait. An **orphan**'s parent terminated (it gets re-parented to init). **Cascading**

termination: some OSes kill all children when the parent exits. The parent may **abort()** a child that exceeds its resources or is no longer needed. The shell does fork → exec → wait.

IPC: cooperating processes (reasons: information sharing, speedup, modularity, convenience). **Shared memory** (fast, the user handles sync) vs **message passing** (the kernel is involved; good for small data or distributed systems). Direct: send(P,msg)/receive(Q,msg), one link per pair, automatic. Indirect: mailboxes/ports, links can have more than 2 processes. Blocking = synchronous. Blocking send + blocking receive = **rendezvous**. Buffering: zero capacity (sender waits), bounded (wait if full), unbounded (never waits). POSIX shm: `shm_open, ftruncate, mmap`.

Client-server: sockets (IP:port; 127.0.0.1 is loopback; server: socket → bind → listen → accept → read/write; client: socket → connect → write/read), RPC, RMI, pipes. **Ordinary pipe**: unidirectional, needs a parent-child relationship, fd[0] = read end, fd[1] = write end (Windows calls these anonymous pipes). **Named pipe** (FIFO): bidirectional, no parent-child needed, several processes can use it.

Threads share code, data, files. Each thread has its own registers, stack, PC. Benefits: **responsiveness, resource sharing, economy, scalability**. **Concurrency** means more than one task makes progress (possible on one core). **Parallelism** means tasks run simultaneously (needs multiple cores). Data parallelism vs task parallelism. Multicore challenges: dividing activities, balance, data splitting, data dependency, testing.

Model	Mapping	Pros / cons, examples
Many-to-one	N user → 1 kernel	Thread management in user space (fast). One block blocks all. No parallelism on multicore. Solaris Green Threads, GNU Portable Threads.
One-to-one	1 user → 1 kernel	More concurrency. Creating a user thread creates a kernel thread, so the number of threads may be limited. Windows, Linux, Solaris 9+.
Many-to-many	N user → M kernel (M ≤ N)	Best of both. Solaris before 9, Windows ThreadFiber.
Two-level	M:M + bound threads	IRIX, HP-UX, Tru64, Solaris 8 and earlier.

Libraries: Pthreads (a POSIX *specification*, user or kernel level), Win32, Java (extend Thread or implement Runnable; managed by the JVM). **Implicit threading:** thread pools (faster than creating a thread per request, bounds the thread count), OpenMP (`#pragma omp parallel for`), Grand Central Dispatch (^{ } blocks with serial and concurrent dispatch queues).

Threading issues: fork/exec semantics (§1.7). **Signals**: synchronous or asynchronous, default or user handler. For a multithreaded process, deliver to the thread the signal applies to, to all threads, to certain threads, or to a designated signal thread. **Cancellation**: asynchronous (immediate) vs **deferred** (the default; the target checks at cancellation points like `pthread_testcancel()`, then runs a cleanup handler). **TLS**: each thread's own copy of data that persists across function calls (unlike locals). **Scheduler activations**: **LWP**s act as virtual processors between user and kernel threads, and the kernel makes **upcalls** to the thread library so it can keep the right number of kernel threads. Linux uses `clone()` with flags CLONE_FS/VM/SIGHAND/FILES, and calls threads "tasks". Windows uses ETHREAD, KTHREAD (kernel space) and TEB (user space).

```
pthread_t tid; pthread_attr_t attr;
pthread_attr_init(&attr);
pthread_create(&tid, &attr, runner, arg); /* void *runner(void*) */
pthread_join(tid, NULL);   pthread_exit(0);
pthread_mutex_lock(&m); ... pthread_mutex_unlock(&m);
```

§5 Main memory reference

Address binding: at **compile time** (absolute code; recompile if the start location changes), **load time** (relocatable code bound by the loader), or **execution time** (the process can move during execution; needs an MMU with relocation and limit registers). Logical and physical addresses are the same for compile-time and load-time binding and **differ for execution-time binding**. **MMU:** physical = logical + relocation register. Protection: trap if logical ≥ limit.

Dynamic loading: a routine isn't loaded until it's called. No OS support is required. **Dynamic linking:** a stub locates the shared library at run time (shared libraries, versioning). **Swapping:** roll out, roll in (backing store). Most of the cost is transfer time: a 100 MB process at 50 MB/s takes 2 s out + 2 s in = 4 s. You can't swap a process with pending I/O (or use double buffering). Mobile systems don't swap (flash).

Contiguous allocation: variable partitions and holes. **First-fit**, **best-fit** (smallest leftover; must search the whole list), **worst-fit** (largest leftover). First-fit and best-fit beat worst-fit in speed and storage use. **External fragmentation:** enough total free memory, but not contiguous. **50-percent rule**: with N blocks allocated, 0.5N are lost, so 1/3 of memory may be unusable. Fixes: **compaction** (needs dynamic relocation) or paging. **Internal fragmentation:** unused space inside an allocated block.

Segmentation: the user's view. Logical address = <segment#, offset>. The segment table holds base and limit. STBR gives the table location, and STLR gives the number of segments (s legal if s < STLR). Check offset < limit, else trap. Protection and sharing happen per segment, but it suffers external fragmentation.

Paging: frames (physical) and pages (logical) of the same power-of-two size. No external fragmentation. Internal fragmentation averages ½ page; the worst case is 1 frame − 1 byte. Example: page 2048 B, process 72,766 B = 35 pages + 1,086 B, so 36 frames and 2048−1086 = **962 B** internal fragmentation. Logical address with 2^m space and 2^n pages: page# = high m−n bits, offset = low n bits. The page table is in memory (PTBR, PTLR), so each access takes **2 memory accesses**. **TLB** (associative, 64–1024 entries, ASIDs avoid flushing on context switch, entries can be wired down) fixes that. A **valid–invalid bit** plus protection bits (r/w/x) give protection. **Shared pages**: reentrant code (one copy of the editor; private data per process).

Page-table structure: 32-bit with 4 KB pages gives 2^{20} entries × 4 B = 4 MB per process. **Hierarchical** (forward-mapped): 32-bit with 1 KB pages splits as p1=12, p2=10, d=10. For 64-bit, 2 levels aren't enough (outer table 2^{42} entries); a 3rd level still leaves 2^{34} B and up to 4 memory accesses. **Hashed**: hash the VPN into a chain of (VPN, frame, next). Clustered tables are good for sparse spaces. **Inverted**: one entry per physical frame, holding (pid, page). It saves memory but searching is slow (use a hash table and TLB), and shared memory is hard (only one mapping per frame). **x86-64:** 48-bit virtual addresses in practice, 4-level paging, pages of 4 KB, 2 MB, or 1 GB, up to 52-bit physical with PAE. **ARM:** 4/16 KB pages and 1/16 MB sections. Two TLB levels: inner (main) first, then outer micro TLBs, then a page walk. **IA-32 PAE:** 3 levels (2-bit pointer table | 9 | 9 | 12), 64-bit entries, 36-bit (64 GB) physical.

§6 Deadlock reference (topic 07)

4 necessary conditions (all must hold at once): **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**. **Resource-allocation graph:** request edge Pi→Rj, assignment edge Rj→Pi. No cycle means no deadlock. A cycle with single instances means deadlock. A cycle with multiple instances means deadlock is *possible*.

Handling: ignore it (most OSes) / prevent or avoid it / detect and recover. **Prevention** (break one condition): ME can't be broken for non-sharable resources. Hold-and-wait: request everything up front, or only when holding nothing (low utilization, starvation). No preemption: release held resources if a request can't be granted. Circular wait: impose a **total order** on resources and request in increasing order (e.g. lock accounts in a fixed order).

Avoidance: needs each process's max claim in advance. **Safe state**: there is a sequence <P1..Pn> where each Pi's remaining need ≤ Available + resources held by all Pj with j<i. Safe means no deadlock.

Unsafe means deadlock is possible. Single instance: RAG with claim edges (dashed); grant a request only if it creates no cycle. Multiple instances: **Banker's** algorithm.

```
Need = Max - Allocation
Safety: Work=Available; Finish[i]=false
  find i: !Finish[i] && Need_i <= Work
      -> Work += Alloc_i; Finish[i]=true; repeat
  all Finish true  -> SAFE
Request_i: if Req_i > Need_i -> error;  if Req_i > Avail -> wait
  pretend: Avail-=Req; Alloc_i+=Req; Need_i-=Req;
  safe ? grant : restore old state & wait
Example: A=10, B=5, C=7 instances; Avail=(3,3,2)
      Alloc  Max    Need
  P0  010    753    743
  P1  200    322    122
  P2  302    902    600
  P3  211    222    011
  P4  002    433    431     safe seq: <P1, P3, P4, P2, P0>
  P1 req (1,0,2): <=Need, <=Avail -> Avail=(2,3,0) -> safe -> grant
```

Detection: wait-for graph (single instance) or a Banker-like algorithm using the current Request. **Recovery:** abort all or one at a time; preempt resources (choose a victim, roll back, avoid starvation).

§7 Basics (topics 01–02)

Interrupt-driven OS: an interrupt transfers control to the ISR via the **interrupt vector**, saving the address of the interrupted instruction. A **trap/exception** is a software interrupt caused by an error or a system call. **DMA:** the device controller moves a whole block to memory and raises one interrupt per block. **Dual mode**: a hardware **mode bit** (kernel=0 / user=1). **Privileged instructions** run only in kernel mode (I/O, timer, interrupt disable). A system call switches to kernel mode, and return resets to user mode. A **timer** interrupt prevents infinite loops and processes hogging the CPU. Multiprocessors need **cache coherency**. UMA vs NUMA. Asymmetric vs symmetric multiprocessing.

System calls are usually reached via an API (Win32, POSIX, Java): portability and simplicity. Each call has a number indexed in a table. Parameters are passed in registers, in a block/table pointed to by a register, or on the stack. Types: process control, file management, device management, information maintenance, communications, protection. printf() → write(). **OS structures:** simple (MS-DOS), monolithic (UNIX), layered, microkernel (Mach; small kernel, message passing, more reliable but slower), loadable modules, hybrid.